

Integrated multi-factory production scheduling and vehicle routing with factory eligibility

Kai Wang^a , Yun Huang^{b,*}

^a Economics and Management School, Wuhan University, Wuhan, China

^b School of Business, Macau University of Science and Technology, Taipa, Macao

ARTICLE INFO

Keywords:

Production scheduling
Vehicle routing
Distribution estimation algorithm
Adaptive large neighborhood search
Q-Learning

ABSTRACT

To enhance the operational performance of supply chains under the trends of globalization and customization, integrated multi-factory production and distribution has recently attracted increasing attention. This paper presents a novel integrated multi-factory production scheduling and vehicle routing problem. In this problem, a set of customer orders is first assigned to several distributed factories for production, each of which is arranged as a hybrid flow shop (HFS). Owing to the technical or physical aspects, factory eligibility is considered in the production stage, where some orders can only be processed in a subset of factories. The finished products are then delivered by capacitated vehicles, subject to customer time windows. As a combination of the distributed HFS scheduling problem and the vehicle routing problem, three types of decisions have to be made, namely factory allocation, job scheduling, and vehicle assignment and routing. Considering the NP-hardness of the studied problem, a hybrid algorithm that integrates a distribution estimation algorithm (EDA) with an adaptive large neighborhood search (ALNS) is developed to generate solutions. To improve the local search capability of this algorithm, Q-Learning is employed to dynamically determine the destroy-and-repair operators of ALNS. Computational results on both small-sized and large-sized test problems indicate the superiority of the proposed algorithm.

1. Introduction

Production and distribution are two essential tasks of manufacturers that are performed separately and sequentially in their manufacturing and logistics departments (Berghman et al., 2023). The trends of high customization and distributed production require a closer integration between these tasks in the make-to-order (MTO) supply chain (Kumar et al., 2020). In practice, factories differ in production capacity and technical capability, and customer orders must be assigned to those that can satisfy specific process and quality requirements. If production and distribution are planned independently, orders may be assigned to distant factories, leading to higher logistics costs and delivery delays. This drawback can be overcome by jointly optimizing production and distribution, which allows simultaneous consideration of factory capabilities and customer locations. Such integration balances production loads, improves delivery performance, and reduces costs for MTO customers, which is especially critical in traditional time-intensive industries such as newspapers, medical products, perishable foods, and fashion apparel and footwear, as well as in high-tech manufacturing

sectors including electronics and automotive components (Hou et al., 2022). Therefore, motivated by these real-world operational challenges, the integrated production and distribution problem (IPDP) has recently received significant attention from both industry and academia (Wang et al., 2025).

A typical IPDP involves two combinatorial optimization problems, namely the machine scheduling problem in the production stage and the vehicle routing problem in the distribution stage (Zhao and Zhou, 2025). The majority of existing works on IPDP focus on production scheduling in a single factory, which is accordingly regarded as the only depot to design vehicle routes for finished product delivery. A variety of manufacturing environments have been considered (Fu et al., 2022) within a factory, including single machine, parallel machines, flow shops, and job shops, etc. Viergutz and Knust (2014) considered a scenario with a single machine environment and multiple customers, where the perishable food was first produced with a limited production rate and then delivered by a single capacitated vehicle within the lifespan or time windows. More recently, Wang et al. (2019) investigated a parallel machine production and distribution model to minimize the total travel

* Corresponding author.

E-mail addresses: kai.wang@whu.edu.cn (K. Wang), yuhuang@must.edu.mo (Y. Huang).

<https://doi.org/10.1016/j.eswa.2026.131291>

Received 25 June 2025; Received in revised form 27 December 2025; Accepted 18 January 2026

Available online 28 January 2026

0957-4174/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Table 1
Summary of the research works on IPDP.

Reference	Factory setting			Shop type					Vehicle routing	Time window or due time	Objectives	Optimization Methods
	SF	MF	FE	SM	PM	PFS	HFS	JS				
Cheng et al. (2017)	✓			✓					✓		Total cost of production and distribution	ACO
Gharaei and Jolai (2019)		✓		✓					✓	✓	Total tardiness and transportation cost	B&C
Li et al. (2023)		✓		✓					✓		Total fixed costs, weighted delivery time, and storage costs	B&B
Wang et al. (2019)	✓				✓				✓	✓	Total travel cost and tardiness cost	MA
Sun et al. (2015)		✓			✓					✓	Total operating costs	GA
Martins et al. (2021)	✓						✓		✓		Makespan	VND
Qiu et al. (2022)	✓						✓		✓		Makespan	MA
Fu et al. (2022)		✓				✓			✓		Makespan	BWO
Hou et al. (2022)		✓				✓			✓		Total weighted earliness and tardiness	BSO
Zhang and Zhou (2024)		✓				✓			✓	✓	Total energy consumption and total earliness/tardiness	AOA
Deng et al. (2023)		✓					✓			✓	Earliness, tardiness, and delivery costs	EDA
Mohammadi et al. (2020)	✓							✓	✓	✓	Production and distribution cost	PSO
Zhao and Zhou (2025)		✓						✓	✓	✓	Total tardiness and total energy consumption	MA
This study		✓	✓				✓		✓	✓	Distribution cost and tardiness cost	EDA, ALNS

SF: single factory; MF: multi-factory; FE: factory eligibility; SM: single machine; PM: parallel machines; PFS: permutation flow shop; HFS: hybrid flow shop; JS: job shop; ACO: ant colony optimization; B&C: branch and cut; B&B: branch and bound; MA: memetic algorithm; GA: genetic algorithm; VND: variable neighborhood descent; BWO: black widow optimization; BSO: brain storm optimization; AOA: arithmetic optimizer algorithm; EDA: distribution estimation algorithm; PSO: particle swarm optimization; ALNS: adaptive large neighborhood search.

cost and tardiness cost. This model considered some complicated realistic features, including machine-dependent ready times, time windows, and uncertain travel times. Motivated by a real case from the furniture industry, Mohammadi et al. (2020) addressed an integrated flexible job-shop scheduling and vehicle routing problem with time windows. Martins et al. (2021) presented a combined hybrid flow shop and vehicle routing problem (HFS-VRP), in which a single capacitated vehicle made multiple trips to distribute finished items. Inspired by the practice of a traditional Chinese medicine decoction company in Shanghai, Qiu et al. (2022) considered a more complicated HFS-VRP with multi-trip multi-vehicle delivery. Owing to the complex machine configuration of flow shops and job shops, research efforts on IPDP have been mainly devoted to relatively simple manufacturing environments, e.g., single machine and parallel machines.

To enhance competitiveness in today's rapidly changing business environment, production has been shifted from a single factory to a multi-factory production network (Toptal and Sabuncuoglu, 2010; Behnamian and Fatemi Ghomi, 2016). Flow shops are common production environments, typically modeled as either a permutation flow shop (PFS) or a hybrid flow shop (HFS). A PFS processes jobs through sequential single-machine stages, whereas an HFS allows multiple machines per stage, thereby offering greater flexibility but also increased complexity (Li et al., 2019; Cai et al., 2022). Despite the prevalence of flow shops in production environments, few studies have investigated flow shop production and distribution problems in a multi-factory environment. Fu et al. (2022) presented an integrated distributed PFS production and distribution problem to minimize the maximum completion time. In this problem, multi-factory production was modeled as a distributed PFS, while distribution from factories to customers was formulated as a multi-depot vehicle routing problem. Hou et al. (2022) also focused on an IPDP in a distributed PFS environment. Unlike Fu et al. (2022), they incorporated customer time windows in the distribution stage and aimed to minimize the total weighted earliness and tardiness. More recently, Zhang and Zhou (2024) proposed a new arithmetic optimizer algorithm to minimize total energy consumption and earliness/tardiness for distributed PFS production and distribution. Extending the distributed PFS to a more complex production environment, Deng et al. (2023) addressed a fuzzy distributed HFS scheduling

considering on-time delivery. They focused on handling fuzzy processing times and due dates to maximize delivery accuracy, while vehicle routing was not taken into account.

A variety of approaches have been proposed for IPDPs, which are mainly divided into three categories, namely exact algorithms, heuristics, and meta-heuristics (Moons et al., 2017). Exact algorithms can only find the optimal solutions for small-sized problems in relatively simple production environments. The branch-and-bound algorithm (Armstrong et al., 2008; Li et al., 2022), the branch-and-price algorithm (Karaoğlu and Kesen, 2017; Gharaei and Jolai, 2019), and the Benders decomposition algorithm (Zhang et al., 2021) have been successfully applied to solve the IPDPs with a single machine or parallel machines. Owing to the NP-hardness of IPDPs, most studies employ heuristics and meta-heuristics to generate near-optimal solutions within reasonable computation time for large-sized IPDPs. Meta-heuristics can be mainly grouped into single solution-based and population-based algorithms. The former is exploitation-oriented and commonly used for VRPs, whereas the latter is exploration-oriented and tends to obtain better schedules for machine scheduling problems. As the combination of machine scheduling problem and VRP, IPDP has therefore been successfully addressed by both single solution-based and population-based meta-heuristics in the literature, mainly including tabu search (TS) (Jamili et al., 2016), iterated local search (ILS) (Fortes et al., 2022), genetic algorithm (GA) (Liu et al., 2021), memetic algorithm (MA) (Wang et al., 2019), ant colony optimization (ACO) (Cheng et al., 2015), particle swarm optimization (PSO) (Mohammadi et al., 2020), black widow optimization (BWO) (Fu et al., 2022), and brain storm optimization (BSO) (Hou et al., 2022), arithmetic optimizer algorithm (Zhang and Zhou, 2024), etc.

To simplify the integration of production and distribution in a multi-factory environment, existing studies generally assume that each factory can handle all jobs or orders. However, factory eligibility exists in many real-world multi-factory systems, indicating that some customer orders cannot be manufactured in all factories due to process capabilities and technological constraints (Qin et al., 2021; Wang et al., 2022). It is observed in various industries, including automotive, electronics, and consumer goods (Xiong et al., 2025). For instance, the production of automotive camera lenses involves five key stages: spacer assembly, lens and infrared assembly, MTF testing, stray light testing, and appearance

inspection. Not all factories are capable of handling every product type. Some specialize in standard lenses with basic assembly and testing lines, while others can produce advanced infrared lenses requiring precision equipment and stricter testing. Consequently, customer orders must be assigned to the factories that meet the required technical and quality standards.

Motivated by the need for distributed production in multi-factory environments with factory eligibility, this paper focuses on a novel multi-factory IPDP, namely distributed HFS (DHFS) scheduling and vehicle routing problem with factory eligibility (DHFSVRP-FE). In this problem, each factory is regarded as an HFS, and customer orders are first assigned and processed in the DHFS with consideration of factory eligibility. Finished products of factories are subsequently delivered by capacitated vehicles to customers with time window constraints. Therefore, DHFSVRP-FE is a combination of the DHFS scheduling problem (DHFSPP) and the multi-depot vehicle routing problem (MDVRP). Considering the NP-hardness of these two combinatorial optimization problems, DHFSVRP-FE is complicated to solve. To highlight the differences between the DHFSVRP-FE and existing IPDPs in the literature, we summarize related studies in Table 1. As shown in this table, no previous studies have attempted to integrate DHFS production and distribution under factory eligibility constraints.

As a new variant of the IPDP, the integration of production scheduling and vehicle routing under factory eligibility constraints in a DHFS environment significantly increases the problem complexity. Accordingly, this study contributes to model formulation and algorithmic design of the novel DHFSVRP-FE. Owing to the NP-hardness of the problem, we propose an effective hybrid algorithm (EDA-QALNS) that integrates a distribution estimation algorithm (EDA) with a Q-learning-based adaptive large neighborhood search (ALNS). In this algorithm, EDA firstly employs the probabilistic model to generate populations, and then ALNS with several destroy and repair operators is applied to further improve some selected good solutions. Although the traditional ALNS adopts the well-known roulette wheel selection method to determine the operators, it tends to be difficult to identify appropriate operators when most of the considered operators have similar probabilities to be selected, especially in the later optimization phase (Li et al., 2020). Therefore, to enhance the local search performance of ALNS, Q-learning is adopted to dynamically determine the destroy and repair operators based on the rewards of their past successful neighborhood search. To the best of our knowledge, there is no research work to hybridize EDA, ALNS, and Q-learning for IPDPs. The proposed EDA-QALNS is expected to help operations managers of manufacturing industries better coordinate the production scheduling and distribution in multi-factory environments.

The remainder of the paper is organized as follows. Section 2 formulates the proposed DHFSVRP-FE. Section 3 presents a novel hybrid algorithm to deal with the DHFSVRP-FE. To validate the effectiveness of the proposed algorithm, experiment results on both small-sized and large-sized DHFSVRP-FEs are discussed in Section 4. Finally, Section 5 concludes the paper and proposes some further research directions.

2. Problem description and formulation

2.1. Problem description

The DHFSVRP-FE with factory eligibility is a novel combinatorial optimization problem. This problem involves two stages: the production stage and the distribution stage. In the production stage, a set of jobs J , each associated with an order from a customer $g \in C$, must be assigned to a set of factories F . An order may comprise one or multiple jobs, which are required to be processed within the same factory. Owing to the technical or quality requirements of customers, factory eligibility has to be considered, and some orders can only be processed in specific factories. Each factory $k \in F$ is arranged as an HFS with s stages, where each stage $l \in S$ consists of a set of parallel machines M_{kl} . The machine

configuration is heterogeneous: the number of machines may differ not only across factories at the same stage but also across stages within the same factory. Once a job $i \in J$ is assigned to factory k , it must be processed sequentially through all stages. At each stage $l \in S$ in factory k , the job is executed on exactly one machine from M_{kl} .

In the distribution stage, a fleet of vehicles V is dispatched to deliver completed jobs from factories to each customer $g \in C$ within its designated time windows $[a_g, b_g]$. Each customer is served by at most one vehicle $r \in V$. After completing its route, each vehicle returns to its origin factory. The delivery cost of a vehicle consists of a fixed part and a variable part proportional to its travel time. If a vehicle arrives at customer g before a_g , it must wait until the service window opens; if it arrives after b_g , it incurs tardiness costs.

To solve the proposed DHFSVRP-FE, three interrelated decisions have to be made simultaneously: (1) allocating customer orders to factories under factory eligibility constraints; (2) scheduling the assigned jobs in each of the HFSs; (3) determining a set of feasible vehicle routes to deliver finished products from factories to customers. To simplify the integration of production and distribution in the distributed HFSs, the following assumptions are made: (1) All factories, whose locations are given in advance, are available at time zero; (2) each order can be allocated to only one eligible factory for processing, where eligibility is determined by the factory's capability to handle the order's required operations; (3) preemption and interruption are not allowed in an HFS; (4) the parallel machines at a stage take equal time to process the same job; (5) there are an infinite number of homogeneous vehicles with the same capacity and fixed cost; (6) the load/unload time of a vehicle is negligible; (7) a customer can only be visited once; (8) both job processing time on machines and vehicle travel time between a factory and a customer or between two customers are deterministic.

2.2. Mathematical formulation

The following notations are used to describe the proposed DHFSVRP-FE.

Sets

C Set of customers, $C = \{1, 2, \dots, c\}$.

J Set of jobs, $J = \{1, 2, \dots, n\}$.

F Set of factories, $F = \{1, 2, \dots, f\}$.

S Set of stages in an HFS, $S = \{1, 2, \dots, s\}$.

M_{kl} Set of parallel machines at stage l in factory k , $M_{kl} = \{1, 2, \dots, m_{kl}\}$.

V Set of vehicles, $V = \{1, 2, \dots, v\}$

Parameters

x_{ig} 1 if job $i \in J$ belongs to the order of customer $g \in C$; and 0 otherwise.

y_{kg} 1 if factory $k \in F$ can process the order of customer $g \in C$; and 0 otherwise.

p_{kij} Processing time of job $i \in J$ at stage $l \in S$ in factory $k \in F$.

JS_i Size of job $i \in J$.

TS_g Total size of jobs of customer $g \in C$.

Q Maximum capacity of vehicles.

FC Fixed cost of a vehicle.

VC Variable cost of a vehicle per unit of time.

t_{kg} Travel time between factory $k \in F$ and customer $g \in C$.

t_{gh} Travel time between customers $g \in C$ and $h \in C$.

$[a_g, b_g]$ Service time window of customer $g \in C$.

TC Tardiness cost per unit of time.

M_1, M_2, M_3 Sufficiently large numbers.

Decision variables

S_{il} Start time of job $i \in J$ at stage $l \in S$.

C_g Completion time of all the jobs of customer $g \in C$.

D_{kr} Departure time of vehicle $r \in V$ at factory $k \in F$.

L_g Leave time of a vehicle at customer $g \in C$.

z_{kg} 1 if the order of customer $g \in C$ is assigned to factory $k \in F$; and 0 otherwise.

α_{ki} 1 if job $i \in J$ is processed in factory $k \in F$; and 0 otherwise.
 β_{kloi} 1 if job $i \in J$ is processed on machine $o \in M_{kl}$ at stage $l \in S$ in factory $k \in F$; and 0 otherwise.
 γ_{kloij} 1 if job $j \in J$ is processed immediately after job $i \in J$ on machine $o \in M_{kl}$ at stage $l \in S$ in factory $k \in F$; and 0 otherwise.
 η_{krig} 1 if the order of customer $g \in C$ are delivered by vehicle $r \in V$ at factory $k \in F$; and 0 otherwise.
 λ_{krgh} 1 if vehicle $r \in V$ at factory $k \in F$ serves customer $g \in C$ followed by customer $h \in C$; and 0 otherwise.

Based on the above notations and the existing models for distributed flow shop scheduling and vehicle routing (Moon et al., 2012; Li et al., 2020), the mathematical model of DHFSSVRP-FE is established as follows:

$$\begin{aligned} \min \quad & FC \sum_{k=1}^f \sum_{r=1}^v \sum_{g=1}^c \lambda_{kr0g} + VC \left(\sum_{k=1}^f \sum_{r=1}^v \sum_{g=1}^c (\lambda_{kr0g} + \lambda_{kr[c+1]}) t_{kg} + \sum_{k=1}^f \sum_{r=1}^v \sum_{g=1}^c \sum_{h=1}^c \lambda_{krgh} t_{gh} \right) \\ & + TC \sum_{g=1}^c \max\{0, L_g - b_g\} \end{aligned} \quad (1)$$

Subject to:

$$z_{kg} \leq y_{kg}, \quad g \in C, k \in F \quad (2)$$

$$\sum_{k=1}^f z_{kg} = 1, \quad g \in C \quad (3)$$

$$\sum_{k=1}^f \alpha_{ki} = 1, \quad i \in J \quad (4)$$

$$\alpha_{ki} = \sum_{g=1}^c x_{ig} z_{kg}, \quad i \in J, k \in F \quad (5)$$

$$\sum_{o=1}^{m_{kl}} \beta_{kloi} = \alpha_{ki}, \quad i \in J, k \in F, l \in S \quad (6)$$

$$\sum_{i=1}^n \gamma_{kloi} = \sum_{i=1}^n \gamma_{kloi[n+1]} \leq 1, \quad k \in F, l \in S, o \in M_{kl} \quad (7)$$

$$\sum_{j=0}^n \gamma_{kloij} = \sum_{j=1}^{n+1} \gamma_{kloij} = 1, \quad i \in J, k \in F, l \in S, o \in M_{kl} \quad (8)$$

$$S_{il} \geq S_{i[l-1]} + \sum_{k=1}^f \alpha_{ki} p_{k[i-1]i}, \quad i \in J, l \in S - \{1\} \quad (9)$$

$$S_{jl} \geq S_{il} + \sum_{k=1}^f \alpha_{ki} p_{kli} - M_1 (1 - \gamma_{kloij}), \quad i, j \in J, k \in F, l \in S, o \in M_{kl} \quad (10)$$

$$C_g \geq S_{is} + \sum_{k=1}^f \alpha_{ki} p_{ksi} - M_1 (1 - x_{ig}), \quad g \in C, i \in J \quad (11)$$

$$\sum_{r=1}^v \eta_{krig} = z_{kg}, \quad g \in C, k \in F \quad (12)$$

$$\sum_{g=1}^{c+1} \lambda_{kr0g} = \sum_{g=0}^c \lambda_{kr[c+1]g} = 1, \quad k \in F, r \in V \quad (13)$$

$$\sum_{g=0}^c \lambda_{krgh} = \sum_{g=1}^{c+1} \lambda_{krhg} = \eta_{krh}, \quad h \in C, k \in F, r \in V \quad (14)$$

$$\sum_{g=1}^c TS_g \eta_{krig} \leq Q, \quad k \in F, r \in V \quad (15)$$

$$D_{kr} \geq C_g - M_1 (1 - \eta_{krig}), \quad g \in C, k \in F, r \in V \quad (16)$$

$$L_g \geq D_{kr} + t_{kg} - M_2 (2 - \lambda_{kr0g} - \eta_{krig}), \quad g \in C, k \in F, r \in V \quad (17)$$

$$L_h \geq L_g + t_{gh} - M_3 (1 - \lambda_{krgh}), \quad g, h \in C, g \neq h, k \in F, r \in V \quad (18)$$

$$L_g \geq a_g, \quad g \in C \quad (19)$$

$$S_{il}, C_g, D_{kr}, L_g \geq 0, \quad g \in C, i \in J, k \in F, l \in S, r \in V \quad (20)$$

$$z_{kg}, \alpha_{ki}, \beta_{kloi}, \gamma_{kloij}, \eta_{krig}, \lambda_{krgh} \in \{0, 1\}, \quad g, h \in C, i, j \in J, k \in F, l \in S, o \in M_{kl}, r \in V \quad (21)$$

The objective function (1) minimizes the sum of distribution cost and tardiness cost of delivering customer orders. The distribution cost includes the fixed cost of vehicles and the variable cost proportional to the travel time. Constraints (2) are factory eligibility constraints. Constraints (3) ensure that the order of a customer should be assigned to the same factory; Constraints (4) guarantee that a job must be processed only in one factory; Constraints (5) indicate that an order and its corresponding jobs are assigned to the same factory; Constraints (6) specify that a job can be processed at only one parallel machine at each stage of an HFS; Constraints (7) introduce two dummy jobs of 0 and $n+1$, which are the first job and the last job on a machine, respectively; Constraints (8) ensure that each job processed on a machine has only one immediately precedent job and only one immediately successive job, respectively; Constraints (9) indicate that a job can be processed on only one machine at a time; Constraints (10) guarantee that a machine can process only one job at a time; Constraints (11) determine the completion time of customer orders at the production stage; Constraints (12) ensure that the finished products of a customer are delivered by a single vehicle; Constraints (13) guarantee that a vehicle starts and ends at the same factory; Constraints (14) specify the flow conservation for each customer; Constraints (15) indicate that the total size of customer orders covered by each trip does not exceed the vehicle capacity; Constraints (16) imply that a vehicle can deliver the products only after they have been completed at the production stage; Constraints (17) and (18) define the leave time of vehicles at customers; Constraints (19) ensure that the vehicle that arrives early at a customer has to wait until the customer time window opens; Constraints (20) and (21) define the domain of decision variables.

In the above mathematical model of DHFSSVRP-FE, the Big-M parameters (M_1 , M_2 , M_3) in Constraints (10), (11), and (16)-(18) must be set appropriately. In this study, the focus is not on determining the tightest possible bounds for these Big-M parameters, but on selecting practically suitable values that ensure the validity of these constraints while maintaining computational tractability. Therefore, M_1 , M_2 , and

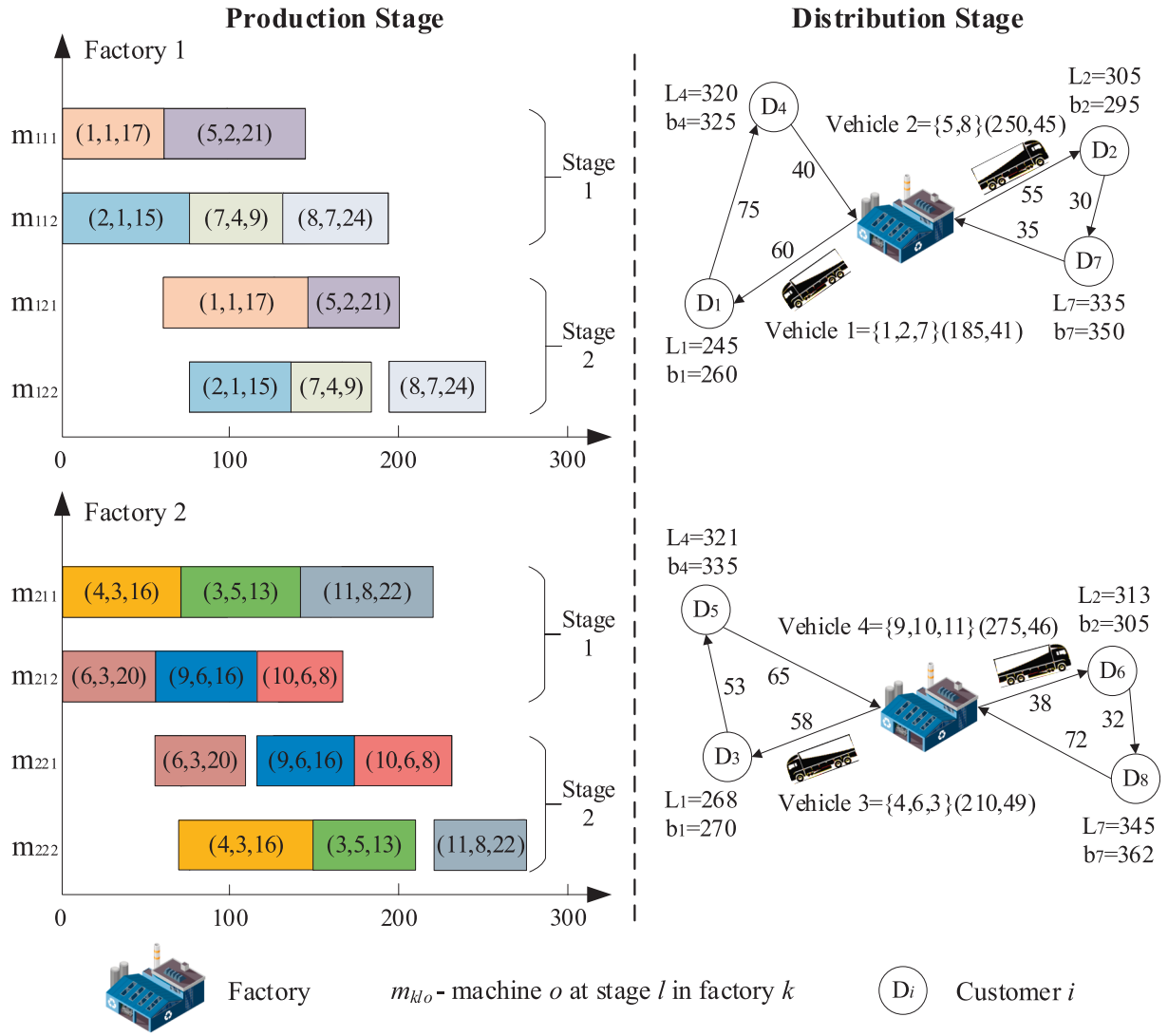


Fig. 1. An example of the DHFSSVRP-FE.

M_3 are defined as $\sum_{l=1}^s \sum_{i=1}^n \max_{k \in F} p_{kli}$, $\sum_{l=1}^s \sum_{i=1}^n \max_{k \in F} p_{kli} + \max_{k \in F, g \in C} t_{kg}$, and $\sum_{l=1}^s \sum_{i=1}^n \max_{k \in F} p_{kli} + c \cdot \max_{k \in F} \{t_{kg}, t_{gh}\}$, respectively.

2.3. Example for the DHFSSVRP-FE

To better illustrate the integrated production and distribution in the DHFSSVRP-FE, we consider an example involving two factories and 11 jobs from 8 customers, where jobs 1 and 2 of customer 1 can only be processed in factory 1. In this example, each factory operates a two-stage HFS, and the capacity limit of a vehicle is 50 units.

In the production stage, each job is represented by a tuple (j, D_j, JS_j) , where j is the job identifier, D_j denotes the customer associated with job j (a customer may have multiple jobs), and JS_j indicates the size of job j . For instance, the tuple $(5, 2, 21)$ corresponds to job 2, which is requested by customer 2 and has a size of 21 units. As shown in Fig. 1, the orders of customers $\{1, 2, 4, 7\}$ and $\{3, 5, 6, 8\}$ are assigned to factories 1 and 2, respectively. The orders of customers $\{1, 2, 4, 7\}$ consist of jobs $\{1, 2, 5, 7, 8\}$, while those of customers $\{3, 5, 6, 8\}$ has jobs $\{3, 4, 6, 9, 10, 11\}$. Once all the jobs of a customer are completed, they can be assigned to a vehicle for delivery, subject to the vehicle's capacity constraint.

In the distribution stage, each vehicle is characterized by a job set

and a tuple (LC_s, TS_s) , where LC_s and TS_s denote the latest completion time and the total size of job set s , respectively. For instance, vehicle 1 = $\{1, 2, 7\}(185, 41)$ indicates that vehicle 1 starts to deliver jobs 1, 2, and 7 at time 185 and their total size is 41 units. For each customer g , the latest service start time (b_g) and the vehicle departure time (L_g) are also provided. As an illustrative example, we focus on the distribution of five jobs from factory 1 to their corresponding four customers. After job set $\{1, 2, 7\}$ is completed at time 150, vehicle 1 first delivers jobs 1 and 2 to customer 1, and then proceeds to customer 4 with job 7, all of which arrive before their latest service times. The remaining job set $\{5, 8\}$ is completed at time 250 and dispatched sequentially to customers 2 and 7, respectively. Since job 5 arrives at customer 2 later than its latest start service time, the tardiness is $L_2 - b_2 = 305 - 295 = 10$. Based on the job scheduling, vehicle assignment, and routing, the total distribution and tardiness costs can finally be obtained.

3. Related algorithms

To solve the proposed DHFSSVRP-FE, EDA, ALNS, and Q-learning are hybridized to generate integrated schedules. Accordingly, this section provides a brief introduction to these three algorithms.

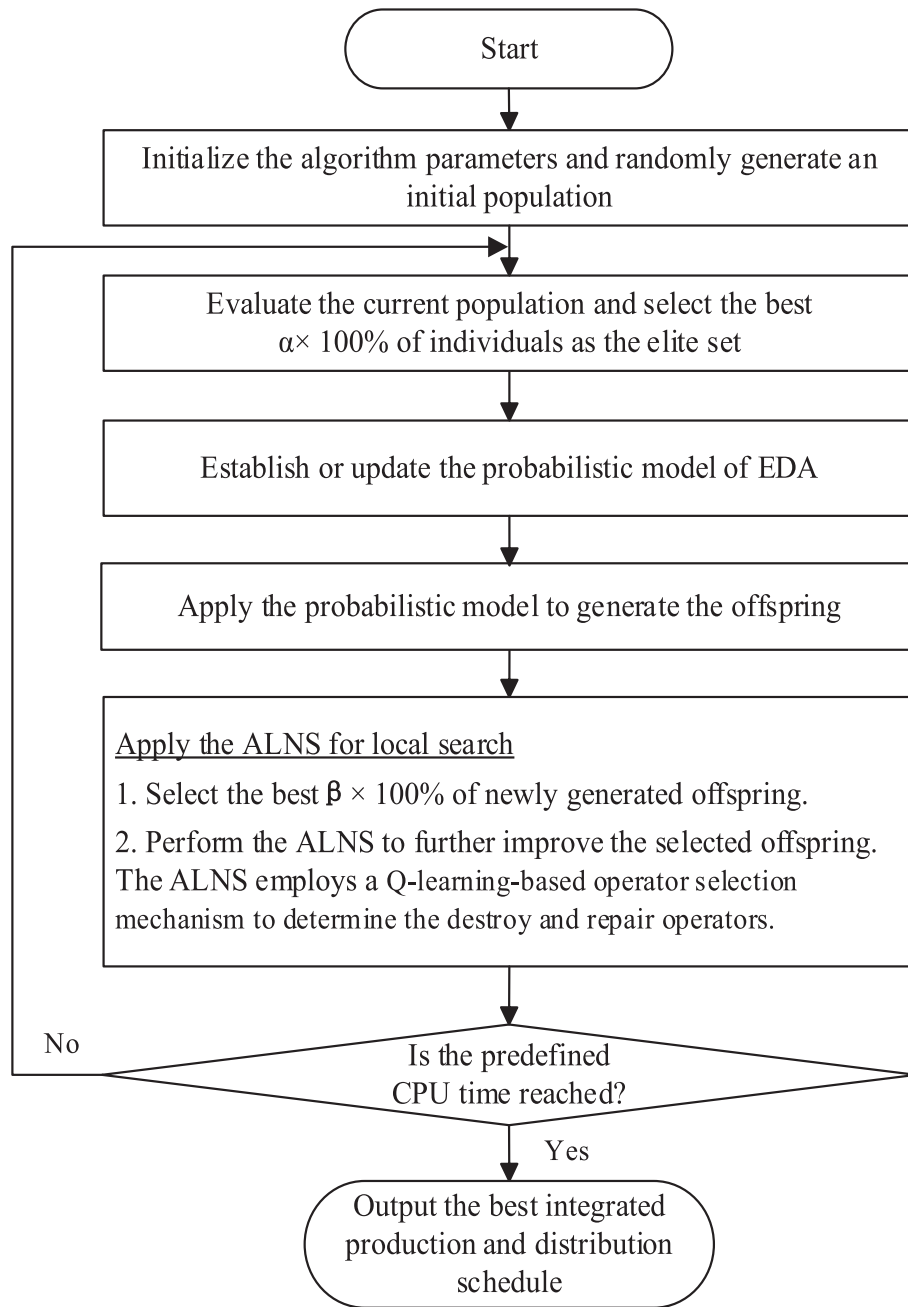


Fig. 2. The framework of EDA-QALNS.

3.1. The standard EDA

EDA, proposed by Mühlenbein and Paass (1996), is a stochastic optimization technique that guides the exploration of the search space by the probabilistic model generated from a set of promising candidate solutions (Hauschild and Pelikan, 2011; Ceberio et al., 2022). Different from traditional evolutionary algorithms, EDA employs a probabilistic model to generate new populations. Since EDA is capable of adapting its probabilistic model to the structure of different problems, it has been successfully applied to either discrete, continuous, or mixed domain problems, including flow shop scheduling problems (Wang et al., 2016).

The standard EDA consists of the following steps:

Step 1: Initialize the algorithm parameters and generate an initial population of solutions.

Step 2: Repeat Steps 3–5 until the stopping criterion is satisfied.

Step 3: Select the promising solutions of the current population and

establish the probabilistic model.

Step 4: Generate the offspring by sampling from the obtained probabilistic model.

Step 5: Update the current population.

Since new populations are generated entirely from the probabilistic model, EDA tends to lose population diversity and get trapped in premature convergence after some generations. Therefore, a trend of integrating EDA with other *meta*-heuristics to solve combinatorial problems has recently been observed, such as genetic algorithm (GA) (Wang et al., 2015), differential evolution (DE) (Zhao et al., 2016), simulated annealing (SA) (Dai et al., 2019), iterated greedy (IG) search (Zheng and Wang, 2020), and variable neighborhood search (VNS) (Du et al., 2021), etc.

Table 2

Job information of customers.

	Customer 1	Customer 2	Customer 3	Customer 4	Customer 5	Customer 6
Job set (order)	{1, 2}	{3, 4, 5}	{6, 7, 8}	{9, 10}	{11, 12, 13}	{14, 15}
Total job size	22	27	38	18	35	21

3.2. The standard ALNS

ALNS, proposed by Ropke and Pisinger (2006), is a relatively new and effective local search algorithm for combinatorial optimization in various industrial domains (Du et al., 2022; Mara et al., 2022), such as manufacturing and production systems, logistics, healthcare, and waste management, etc. As an extension of the large neighborhood search (LNS) (Shaw, 1998), ALNS applies several competing destroy and repair operators to adaptively generate new solutions. Based on the performance of each operator, an adaptive probabilistic mechanism is adopted to dynamically select pairs of destroy-and-repair operators for neighborhood search.

The main steps of the standard ALNS are as follows:

Step 1: Initialize the algorithm parameters and generate a feasible solution S .

Step 2: Repeat Steps 3–5 until the stopping criterion is satisfied.

Step 3: Select a pair of destroy-and-repair operators using roulette wheel selection and reconstruct a new solution S' .

Step 4: If S' is better than S , replace the current solution S with S' and record the best solution found so far; Otherwise, update S using the Metropolis criteria.

Step 5: Update the selection probabilities of destroy and repair operators using their historical and current performance.

Compared with population-based *meta*-heuristics, single solution-based *meta*-heuristics are generally successful in intensifying the local search for neighborhood solutions, and therefore preferred as solution approaches to generate vehicle routes for VRP and its variants (Elshaer and Awad, 2020). As an efficient single solution-based *meta*-heuristic, ALNS has recently been widely used to address the capacitated VRP and the VRP with time windows.

3.3. The standard Q-learning algorithm

Q-learning, first proposed by Watkins and Dayan (1992), is one of the most well-known reinforcement learning algorithms. It employs a Q-table to record the action–value estimates (Q-values) for all state–action pairs. The Q-values are updated iteratively according to the feedback obtained from the environment, enabling the algorithm to approximate the long-term return of actions without relying on an explicit model of the environment. By updating the Q-table, Q-learning converges toward the optimal action-selection policy (Mazyavkina et al., 2021). Owing to its simplicity and effectiveness, Q-learning has been widely used to improve the performance of optimization algorithms, particularly for parameter tuning and adaptive operator selection (Shakya et al., 2023).

The main steps of the Q-learning algorithm are as follows:

Step 1: Initialize the Q-table with zeros.

Step 2: Repeat Steps 3–5 until the stopping criterion is satisfied.

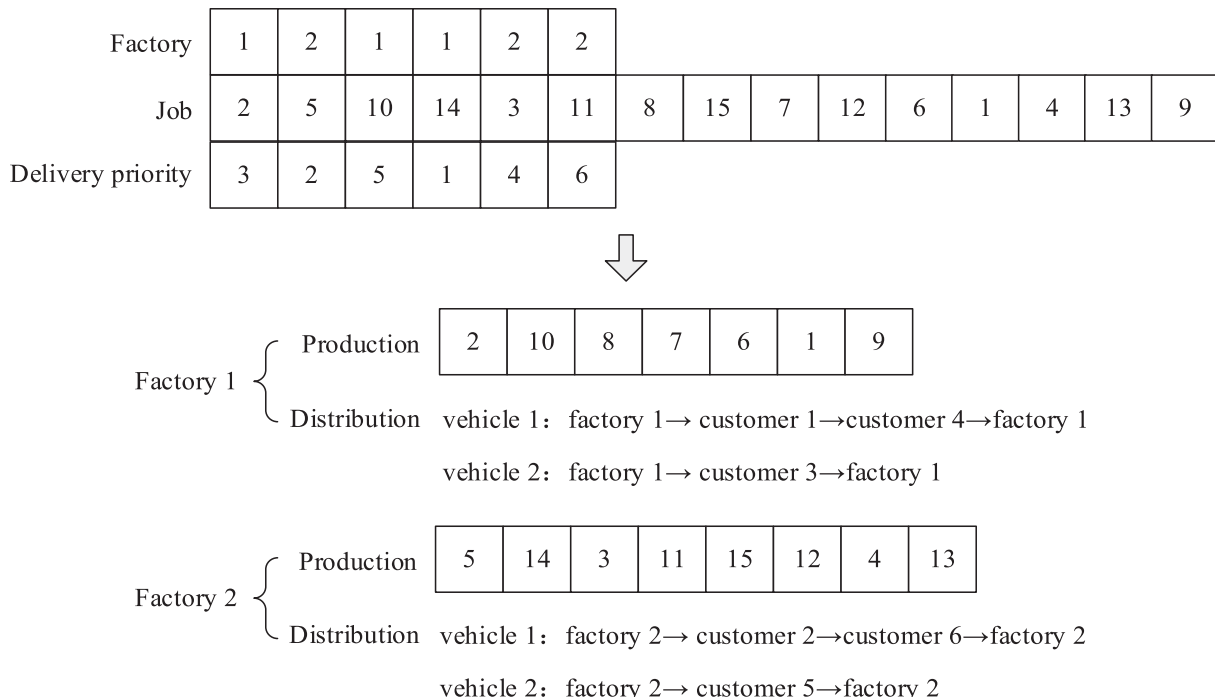
Step 3: Choose an action according to an exploration strategy based on the current state.

Step 4: Execute the selected action, receive the immediate reward, and transition to the next state.

Step 5: Update the Q-values in the Q-table using the Bellman update rule.

4. The proposed EDA-QALNS

This section presents the details of the proposed EDA-QALNS for solving the DHFSSVRP-FEs. As a population-based *meta*-heuristic, EDA-QALNS firstly applies the probabilistic model of EDA to generate the offspring individuals and then adopts the ALNS to further improve some promising ones. Instead of using the traditional roulette wheel selection, a more effective adaptive mechanism is established for the

**Fig. 3.** A chromosome of the proposed DHFSSVRP-FE.

neighborhood search of ALNS, in which Q-learning is employed to dynamically determine the pairs of destroy-and-repair operators. By combining the advantages of EDA and ALNS, EDA-QALNS is capable of providing a good balance between exploration and exploitation. The framework of EDA-QALNS is presented in Fig. 2 and the details of EDA-QALNS are described in the following subsections.

4.1. Encoding and decoding schemes

To generate the integrated schedules of the DHFSSVRP-FE, the three decisions of factory assignment, job scheduling, and vehicle routing have to be integrated. Accordingly, a matrix of three rows is applied to represent a solution (chromosome) of the DHFSSVRP-FE. The second row in the matrix is a permutation of n jobs from c customer orders, which indicates their processing sequence in a multi-factory environment. For each customer, the first row and the third row show its assigned factory under the constraint of factory eligibility and customer delivery priority when its order (set of jobs) is completed, respectively. Thus, the decoding scheme has two steps: (1) job schedules of factories in the production stage are generated based on the factory assignment and the job sequence; (2) vehicles and their delivery routes at the distribution stage are determined based on the customer delivery priorities and the generated job schedules. Following the customer delivery priorities, a vehicle loads all the finished jobs of a customer one by one until the maximum capacity (Q) is met, and its route is accordingly constructed by incrementally inserting the customers whose orders have been loaded. Since a customer can only be visited once, all the finished jobs of a customer have to be loaded in the same vehicle. If the total job size of a customer exceeds the remaining capacity of a vehicle, all its jobs have to be delivered by another vehicle. The departure time of a vehicle depends on the latest completion time of jobs delivered by the vehicle.

To illustrate the decoding of a chromosome, a simple DHFSSVRP-FE is considered as an example. In this problem, 15 jobs from 6 customers need to be processed in two distributed factories, and jobs 9 and 10 of

jobs of customer 5 are finally delivered by vehicle 2. The customer delivery sequence in a route is consistent with the customer delivery priorities shown in the third row of the chromosome.

4.2. The probabilistic model of EDA

EDA-QALNS explores the search space of candidate solutions by sampling the probabilistic model established from promising solutions of the current population. Since a feasible solution is represented by a three-row matrix introduced in Section 4.1, the probabilistic model of EDA-QALNS employs three probability matrices, namely the factory matrix (A), job matrix (B), and delivery priority matrix (C), to describe the statistical information of factory assignment, job sequence, and delivery priority for solution generation, respectively.

At the t^{th} iteration, let $A_{gk}(t)$, $B_{ip}(t)$, and $C_{gq}(t)$ be the probability that the jobs of customer g are assigned to factory k ($1 \leq k \leq f$), the probability that the processing priority of job i is p ($1 \leq p \leq n$), and the probability that the delivery priority of customer g is q ($1 \leq q \leq c$), respectively. To maintain the population diversity of EDA-QALNS, $A_{gk}(t)$, $B_{ip}(t)$, and $C_{gq}(t)$ are initialized uniformly as follows:

$$A_{gk}(0) = 1/f, g \in C, k \in F \quad (22)$$

$$B_{ip}(0) = 1/n, i \in J \quad (23)$$

$$C_{gq}(0) = 1/c, g \in C \quad (24)$$

The traditional incremental learning scheme is applied to update the probabilistic model. Based on the best $\alpha \times 100\%$ ($\alpha \in (0, 1)$) of solutions at the current iteration, $A_{gk}(t)$, $B_{ip}(t)$, and $C_{gq}(t)$ are updated as follows:

$$A_{gk}(t+1) = (1 - r_{EDA})A_{gk}(t) + r_{EDA} \times \frac{1}{SI} \sum_{m=1}^{SI} \eta_{gkm}, g \in C, k \in F \quad (25)$$

$$\eta_{gkm} = \begin{cases} 1, & \text{if customer } g \text{ is assigned to factory } k \text{ of the } m\text{th best solution} \\ 0, & \text{otherwise} \end{cases} \quad (26)$$

customer 4 can only be processed in factory 1. Each of the factories has two vehicles with $Q = 50$ to deliver the completed products to the customers. Based on the customer-job relation and total job size of customers presented in Table 2, Fig. 3 describes a feasible chromosome structure for the DHFSSVRP-FE. As shown in this figure, the orders of customers {1, 3, 4} and {2, 5, 6} are assigned to factory 1 and factory 2 under the constraint of factory eligibility, respectively. Accordingly, jobs {2, 10, 8, 7, 6, 1, 9} and {5, 14, 3, 11, 15, 12, 4, 13} are sequentially produced in these two factories. Considering both the customer delivery priorities and the total job size of each customer, the finished jobs are loaded into vehicles for delivery. We take the distribution of finished jobs in factory 2 as an example and let TS_g represent the total job size of customer g . Among customers 2, 5, and 6, customer 2 has the highest delivery priority and its jobs with $TS_2 = 27$ are first assigned to vehicle 1. The remaining capacity of this vehicle is $50 - 27 = 23$, which cannot allow for loading the jobs of customer 5 with $TS_5 = 35$. Therefore, the jobs of customer 6 with $TS_6 = 21$ are then assigned to vehicle 1, and the

$$B_{ip}(t+1) = (1 - r_{EDA})B_{ip}(t) + r_{EDA} \times \frac{1}{SI} \sum_{m=1}^{SI} \lambda_{ipm}, i \in J \quad (27)$$

$$\lambda_{ipm} = \begin{cases} 1, & \text{if job } i \text{ appears on position } p \text{ of the } m\text{th best solution} \\ 0, & \text{otherwise} \end{cases} \quad (28)$$

$$C_{gq}(t+1) = (1 - r_{EDA})C_{gq}(t) + r_{EDA} \times \frac{1}{SI} \sum_{m=1}^{SI} \mu_{gqm}, g \in C \quad (29)$$

$$\mu_{gqm} = \begin{cases} 1, & \text{if customer } g \text{ appears on position } q \text{ of the } m\text{th best solution} \\ 0, & \text{otherwise} \end{cases} \quad (30)$$

where r_{EDA} is the learning rate, and SI is the number of selected best

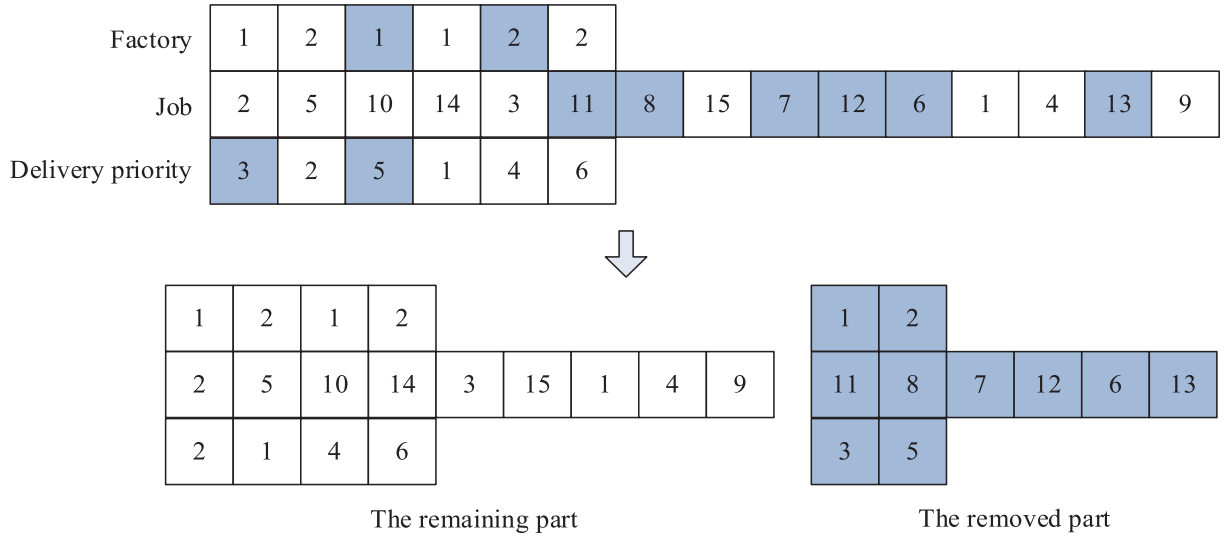


Fig. 4. An example of a destroy operation.

solutions at the current iteration.

According to the above three probabilistic matrices, a feasible solution with a three-row matrix can be generated as follows:

Algorithm 1. Solution generation using the probabilistic model of EDA

Input: the best $\alpha \times 100\%$ ($\alpha \in (0, 1)$) of the solutions at the current iteration
Output: a feasible solution for DHFSSVRP-FE

```

1: // generate the first row for factory assignment
2: for  $g = 1$  to  $f$ 
3:  $r = \text{rand}()$ ; // generate a uniformly random number from  $[0, 1]$ 
4: if  $r < \delta$ 
5:  $r_1(g) = \text{FactoryAssignment}()$ ; // randomly assign an available factory to customer  $g$ 
6: else
7:  $r_1(g) = \text{SelectFactory}()$ ; // assign the factory with the largest  $A_{ip}(t)$  to customer  $g$ 
8: end if
9:  $\text{UpdateFactoryMatrix}()$ ; // remove the elements related to customer  $g$  in the factory matrix
10:  $g = g + 1$ ;
11: end for
12: // generate the second row to determine the job sequence
13:  $\pi_1 = \text{GenerateJobPermutation}()$ ; // generate a random permutation of jobs
14: for  $p = 1$  to  $n$ 
15:  $r = \text{rand}()$ ;
16: if  $r < \delta$ 
17:  $r_2(p) = \pi_1(1)$ ; // put the first job of  $\pi_1$  at position  $p$  in the second row
18: else
19:  $r_2(p) = \text{SelectJob}()$ ; // put the job with the largest  $B_{ip}(t)$  at position  $p$  in the second row
20: end if
21:  $\text{UpdateUnassignedJobs}(\pi_1)$ ; // remove the assigned job at position  $p$  from  $\pi_1$ 
22:  $\text{UpdateJobMatrix}()$ ; // remove the elements related to the assigned job in the job matrix
23:  $p = p + 1$ ;
24: end for
25: // generate the third row to determine the delivery priority
26:  $\pi_2 = \text{GenerateCustomerPermutation}()$ ; // generate a random permutation of customers
27: for  $q = 1$  to  $c$ 
28:  $r = \text{rand}()$ ;
29: if  $r < \delta$ 
30:  $r_3(q) = \pi_2(1)$ ; // put the first customer of  $\pi_2$  at position  $q$  in the third row
31: else
32:  $r_3(q) = \text{SelectCustomer}()$ ; // put the customer with the largest  $C_{gq}(t)$  at position  $q$ 
33: end if
34:  $\text{UpdateUnassignedCustomers}(\pi_2)$ ; // remove the assigned customer at position  $q$  from  $\pi_2$ 
35:  $\text{UpdateDeliveryPriorityMatrix}()$ ; // remove the elements related to the assigned customer
36:  $q = q + 1$ ;
37: end for
38: return the three-row matrix of a solution

```

4.3. The destroy and repair operators of ALNS

To perform better exploitation of solutions, EDA-QALNS adopts destroy and repair operators of ALNS to further improve the best $\beta \times 100\%$ ($\beta \in (0, 1)$) of the new solutions generated by the probabilistic model of EDA. To avoid getting trapped in the local optimum, the widely used Metropolis criterion, also known as the simulated annealing criterion, is adopted to decide whether a new solution is accepted as the incumbent solution.

Considering the three-row encoding scheme presented in Section 4.1, the job sequence in the second row and the delivery priority in the third row have to be updated simultaneously if some customers in the first row are removed in the destroy phase. According to the job information of customers in Table 2, Fig. 4 illustrates an example of a destroy operation, in which the assigned factories, jobs, and delivery priorities of customers 3 and 5 are removed.

Based on the characteristics of the proposed DHFSSVRP-FE, six destroy operators are developed by removing several customers, which are detailed as follows:

- **Random destroy (D1):** This operator randomly removed m customers from a solution.
- **Worst destroy (D2):** The D2 operator removes the customer, giving the greatest saving in the total cost of delivery and tardiness. The cost saving is defined as $\Delta(g, s) = f(s) - f_{-g}(s)$, where $f(s)$ and $f_{-g}(s)$ are the total cost of solution s with and without customer g , respectively.
- **Random worst destroy (D3):** This operator first sorts all the c customers in descending order of $\Delta(g, s)$ and then randomly removes the $\lfloor \gamma \times m \rfloor$ customers from the first m customers. γ is the predefined selection portion of customers.
- **Time-related destroy (D4):** The D4 operator first randomly removes customer g and then removes customer h with the lowest time-relatedness to customer g . The time-relatedness of two customers is defined as $r_{gh}^{\text{time}} = |a_g - a_h| + |b_g - b_h|$, where $[a_g, b_g]$ and $[a_h, b_h]$ are the service time windows of customers g and h , respectively.
- **Tardiness-related destroy (D5):** This operator repeatedly removes the customer with the largest tardiness until m customers have been removed. The tardiness of customer g is measured as $T_g = L_g - b_g$.
- **Distance-related destroy (D6):** The D6 operator first randomly selects a customer and then removes it and its nearest neighbor in terms of the Euclidean distance. The operator iterates until m customers have been removed.

After the destroy phase, three node lists of factories, jobs, and customers are generated, and six repair operators are employed to generate new solutions. These operators have a similar procedure, which consists of four steps: (1) randomly select a removed customer and reinsert its assigned factory in the first row; (2) reinsert the jobs of the selected customer in the second row; (3) reinsert the selected customer in the third row; (4) repeat steps 1–3 until all the removed customers have been reinserted. The six repair operators apply different job and customer insertion schemes in Steps 2 and 3, which are detailed as follows:

- Random repair (R1): This operator randomly inserts the jobs and customers from the generated node lists.
- Greedy repair (R2): The R2 operator decides the best positions for removed jobs and customers based on the lowest incremental cost. Let Δ_{ij}^1 and Δ_{gh}^2 define the completion time increment when putting job i at position j and the increment of delivery and tardiness cost when putting customer g at position h , respectively. The R2 operator iteratively selects and inserts a job at the position with the minimum Δ_{ij}^1 until all the removed jobs have been inserted. Similarly, Δ_{gh}^2 is applied to guide the insertion of removed customers.
- Regret repair (R3): This operator considers a regret criterion for the insertion of jobs and customers. The regret value of job i is measured by the difference between the minimum and the second minimum Δ_{ij}^1 . Similarly, the regret value of customer g is defined by the difference between the minimum and the second minimum Δ_{gh}^2 . The R3 operator iteratively selects and inserts a job or a customer that has the largest regret value.
- Greedy repair with noise (R4): Based on the R2 operator, this operator introduces the noise to randomize the insertion of jobs and customers. The new Δ_{ij}^1 is computed as $\Delta_{ij}^3 = \Delta_{ij}^1 + \bar{p}\mu\epsilon$, where $\bar{p}$ represents the maximum total job processing time of customers, μ denotes the noise constant, and ϵ is a random number between $[-1, 1]$. Similarly, the new Δ_{gh}^2 is defined as $\Delta_{gh}^4 = \Delta_{gh}^2 + \bar{d}\mu\epsilon$, where $\bar{d}$ indicates the maximum delivery cost between two customers or between a customer and its assigned factory.
- Regret repair with noise (R5): This operator is similar to the R4 operator but considers the same noise function as the R3 operator instead.
- Sequential repair (R6): The R6 operator sequentially inserts the removed jobs and customers at the positions with the minimum Δ_{ij}^1 and Δ_{gh}^2 , respectively.

4.4. The Q-learning-based operator selection mechanism

According to the proposed six destroy operators and six repair operators, a total of 36 ($6 \times 6 = 36$) pairs of destroy-and-repair operators can be employed to further improve the solutions generated by EDA. Instead of using the well-known roulette wheel selection method, the Q-learning-based operator selection mechanism (QOSM) is applied to better determine the pair of destroy-and-repair operators for local search, even in the situation where there is not much performance difference between the best and the worst pairs.

In the proposed QBSM, the state space is related to the number of consecutive local search phases with no improvement of the best solution (N), which is defined as follows:

$$S = \begin{cases} 1, & 0 < N \leq 5 \\ 2, & 5 < N \leq 10 \\ 3, & 10 < N \leq 15 \\ 4, & 15 < N \leq 20 \\ 5, & N \geq 20 \end{cases} \quad (31)$$

For each state of local search, there is a set of possible actions that can be performed to improve the best solution found so far. The action

set includes 36 pairs of destroy-and-repair operators. By covering all possible states and actions, the Q-table, i.e., a 5×36 matrix, is established, in which Q values indicate the cumulative reward of state-action pairs. The QBSM applies the traditional ϵ -greedy policy to determine actions. In this policy, a pair of destroy-and-repair operators is randomly selected if a random number is less than the predefined ϵ . Otherwise, the pair of destroy-and-repair operators with the highest reward is chosen. After the selected action is executed, an immediate reward is received and the corresponding cumulative reward in the Q-table is updated. These two types of rewards are computed as follows:

$$r(s_t, a_t) = \begin{cases} r_1, & \text{if the new solution is the new best solution} \\ r_2, & \text{if the new solution is only better than the current solution} \\ r_3, & \text{if the new solution is only better than the worst solution of the current population} \\ r_4, & \text{if the new solution is the worst in the current population and never been found} \\ r_5, & \text{if all the conditions above are not satisfied} \end{cases} \quad (32)$$

$$Q_{t+1}(s_t, a_t) = Q(s_t, a_t) + r_Q[r(s_t, a_t) + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (33)$$

where $r_1 \sim r_5$ are immediate rewards with values $r_1 > r_2 > r_3 > r_4 > r_5$, s_t and a_t represent the current state and the selected pair of destroy-and-repair operators at decision point of t , $r(s_t, a_t)$ and $Q(s_t, a_t)$ denote the immediate reward and cumulative reward obtained at decision point t , r_Q is the learning rate within $[0, 1]$, and γ is the discount factor within $[0, 1]$. For a better understanding of the proposed QOSM, its main steps are illustrated in Algorithm II.

Algorithm II: The Q-learning-based operator selection

Input: $s_t, \epsilon, r_Q, \gamma$, and Q-table

Output: a pair of destroy-and-repair operators, current state, and Q-table

```

1:  $r = \text{rand}()$ ; // generate a uniformly random number from  $[0, 1]$ 
2: if  $r < \epsilon$ 
3:  $a_t = \text{RandomSelection}()$ ; // randomly select a pair of destroy-and-repair operators
4: else
5:  $a_t = \text{QLearningSelection}(s_t)$ ; // select a pair of operators with the maximum Q value
6: end if
7:  $r(s_t, a_t) = \text{Reward}(s_t, a_t)$ ; // Compute the immediate reward using Eq. (32)
8:  $\text{UpdateQTable}(r(s_t, a_t), r_Q, \gamma)$ ; // Update the Q-table using Eq. (33)
9:  $s_{t+1} = \text{UpdateState}()$ ; // Update the current state
10: return  $a_t, s_{t+1}$ , and  $Q_{t+1}$ 

```

5. Experiment results and discussion

To evaluate the effectiveness of EDA-QALNS for solving the novel DHFSSVRP-FE, we compared it with CPLEX and several competitive meta-heuristics, namely EDA, ALNS, and EDA-ALNS. All the algorithms were coded using C++ and implemented on a laptop with Intel® Core™ i5-5200U 2.20 GHz CPU and 8 GB RAM under the Microsoft Windows 10 operating system. This section presents the details of the experiment design, the parameter tuning of EDA-QALNS, and experiment results on different sizes of test sets, respectively.

5.1. Experiment design

Since the DHFSSVRP-FE with factory eligibility is a new combinatorial optimization problem, there are no available benchmark instances in the literature. The problem size of DHFSSVRP-FE is mainly related to the number of factories (f), the number of jobs (n), and the number of customers (c). In the literature on IPDPs, test problem parameters are often generated based on a random uniform distribution to ensure diversity of test instances, making it possible to evaluate the algorithm's performance under different scenarios (Fortes et al., 2022). Based on the

Table 3
Parameter levels of EDA-QALNS.

Parameter Level	Parameters				
	PS	r_{EDA}	α	r_Q	β
1	50	0.05	0.10	0.60	0.10
2	60	0.10	0.15	0.70	0.15
3	70	0.15	0.20	0.80	0.20
4	80	0.20	0.25	0.90	0.25

Table 4
Orthogonal array $L_{16}(4^5)$ and RV.

Trial	Parameter level					RV
	PS	r_{EDA}	α	r_Q	β	
1	1	1	1	1	1	3.61
2	1	2	2	2	2	2.26
3	1	3	3	3	3	3.15
4	1	4	4	4	4	3.27
5	2	1	2	3	4	2.02
6	2	2	1	4	3	2.18
7	2	3	4	1	2	3.95
8	2	4	3	2	1	4.72
9	3	1	3	4	2	4.08
10	3	2	4	3	1	4.24
11	3	3	1	2	4	1.62
12	3	4	2	1	3	1.31
13	4	1	4	2	3	4.02
14	4	2	3	1	4	3.39
15	4	3	2	4	1	3.99
16	4	4	1	3	2	3.27

Table 5
Experiment results of CPLEX and EDA-QALNS for small-sized DHFSSVRP-FEs.

Problem No.	Problem size $f \times n \times c$	CPLEX			EDA-QALNS	
		UB	LB	CPU(s)	Objective	CPU(s)
S01	$2 \times 8 \times 4$	58.25	58.25	0.55	58.25	0.31
S02	$2 \times 8 \times 6$	98.23	98.23	3.54	98.23	2.24
S03	$2 \times 10 \times 4$	56.42	56.42	9.87	56.42	3.83
S04	$2 \times 10 \times 6$	96.86	96.86	61.23	96.86	6.82
S05	$2 \times 12 \times 4$	77.31	77.31	285.76	77.31	0.95
S06	$2 \times 12 \times 6$	103.65	103.65	494.10	103.65	12.81
S07	$3 \times 12 \times 6$	73.47	73.47	25.85	73.47	1.23
S08	$3 \times 12 \times 9$	128.42	128.42	60.33	128.42	13.36
S09	$3 \times 15 \times 6$	70.61	70.61	142.78	70.61	5.67
S10	$3 \times 15 \times 9$	91.73	91.73	912.52	91.73	8.05
S11	$3 \times 18 \times 6$	61.69	61.69	1823.51	61.69	22.45
S12	$3 \times 18 \times 9$	144.72	138.45	3600.00	142.81	26.59
S13	$4 \times 16 \times 8$	74.35	74.35	150.23	74.35	12.80
S14	$4 \times 16 \times 12$	125.12	125.12	1397.57	125.12	19.26
S15	$4 \times 20 \times 8$	73.71	73.71	1137.52	73.71	12.25
S16	$4 \times 20 \times 12$	137.46	125.69	3600.00	135.53	18.93
S17	$4 \times 24 \times 8$	80.76	80.76	1365.72	80.76	15.92
S18	$4 \times 24 \times 12$	147.14	98.56	3600.00	129.22	29.16

work of Ullrich (2013), a set of 18 small-sized problems and a set of 27 large-sized problems, shown in Tables 5 and 6, are randomly generated to evaluate the performance of EDA-QALNS.

For each of the test problems, all the customers and factories are assumed to be distributed in a square area with a length of $\lfloor \rho(c/f) \rfloor$, where the floor function $\lfloor \cdot \rfloor$ indicates the largest integer no more than $\rho(c/f)$. We use the proportion of restricted jobs (p) to describe the factory eligibility. In the production stage, a proportion of jobs $p = 0.2$ are randomly selected to be processed only in a subset of factories, whose number is drawn from a discrete uniform distribution $U(1, f)$. The remaining jobs can be processed in any factory. For each job i , p_{kli} and JS_i are uniformly generated within the intervals $[1, \rho]$ and $[1, \sum_{k=1}^f \sum_{l=1}^s p_{kli}/f]$, respectively. For small-sized DHFSSVRP-FEs, both

s and m_{kl} follow the same uniform distribution within the interval $[2, 3]$, while for large-sized DHFSSVRP-FEs, their distributions shift to include the intervals $[4, 6]$ and $[3, 4]$, respectively. To ensure a vehicle has enough capacity to deliver the finished order with the largest size, Q is set to $(1+r)\max\{TS_g\}$, where r is a uniformly random number between 0 and 1. To avoid infeasible test problems, the lower and upper time limits depend on the job processing time at factories and travel time between a customer and its eligible factories, which are computed as follows:

$$a_g = \sum_{i=1}^n \sum_{l=1}^s \sum_{k=1}^f p_{kli} x_{ig} y_{kg} / f + \max_{k \in F} \{t_{kg} y_{kg}\} + \pi_g \quad (34)$$

$$b_g = a_g + \lambda_g \quad (35)$$

In the above equations, π_g and λ_g are uniformly distributed within the intervals $[0, \delta_1 \rho n / (f + \min\{m_{kl}\})]$ and $[0, \delta_2 \rho]$ respectively, where both δ_1 and δ_2 are adjustment parameters.

5.2. Parameter tuning for EDA-QALNS

The effectiveness of EDA-QALNS depends on the values of key algorithm parameters, mainly including the population size (PS), the learning rate to update the probabilistic model (r_{EDA}), the population ratio to update the probabilistic model (α), the learning rate to update the Q-table (r_Q), and the population ratio for destroy and repair operations (β). Since the Taguchi method is an effective and efficient method for experiment design, it is applied to tune the parameters of EDA-QALNS. The factor levels of these five parameters and the corresponding orthogonal array $L_{16}(4^5)$ are presented in Tables 3 and 4, respectively.

The Taguchi method is performed on three large-sized DHFSSVRP-FEs, namely L07, L14, and L21 shown in Table 6. For each of the problems, EDA-QALNS with a specific parameter combination runs 20 times, and the performance deviation (PD) between the obtained solutions and the best solution is computed as

$$PD = \frac{1}{20} \sum_{r=1}^{20} \frac{Z_r - Z^*}{Z^*} \times 100 \quad (36)$$

where Z_r and Z^* are the objective value obtained in the r th replication and the best objective value over 20 replications, respectively. Accordingly, the average PD of the three test problems is obtained as the value of the response variable (RV), which is also presented in Table 4.

According to the RVs in Table 4, Fig. 5 shows the trends of the five algorithm parameters. Since EDA-QALNS tends to provide better performance when its parameters are set to the values with the minimum RV, the parameter setting of EDA-QALNS is recommended as $PS = 70$, $r_{EDA} = 0.10$, $\alpha = 0.15$, $r_Q = 0.60$, and $\beta = 0.25$.

5.3. Experiment results for small-sized DHFSSVRP-FEs

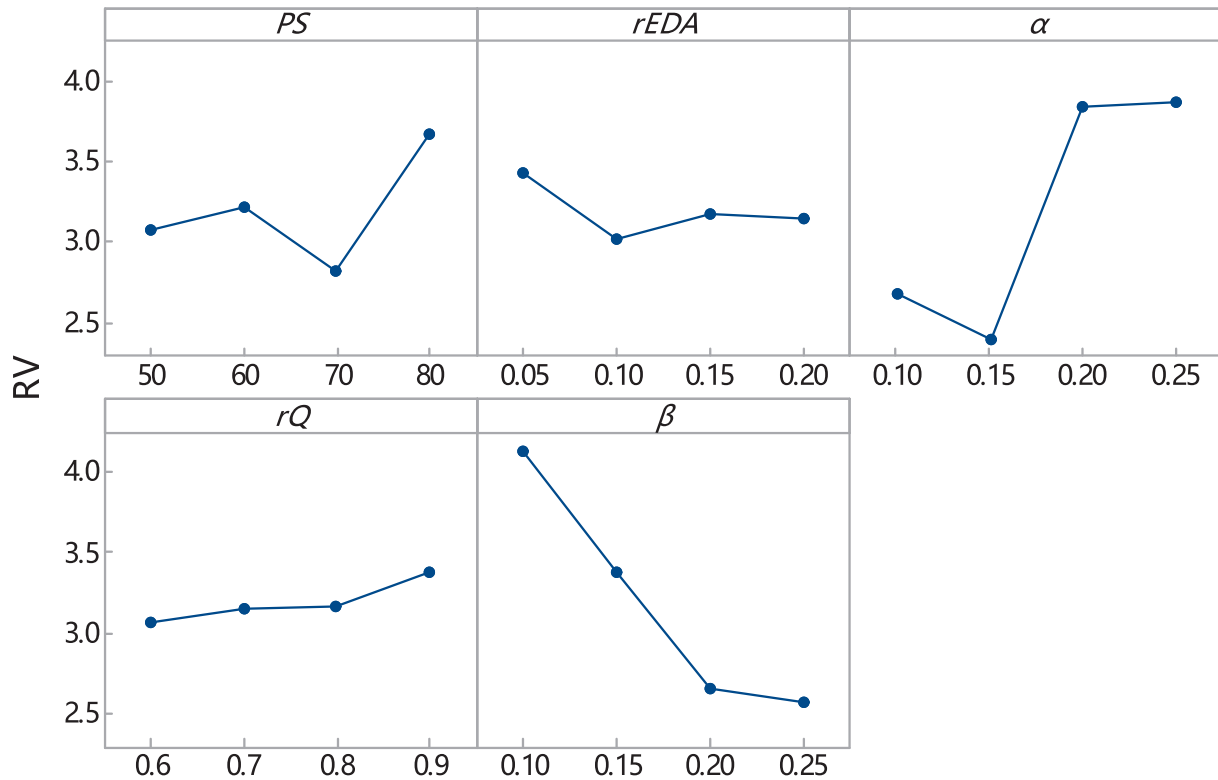
To validate the effectiveness of the proposed EDA-QALNS, the mathematical model of DHFSSVRP-FE is addressed by IBM ILOG CPLEX 12.8 with a time limit of 3600 s. Table 5 compares the experiment results of CPLEX and EDA-QALNS for small-sized problems, where $f = \{2, 3, 4\}$, $n/f = \{4, 5, 6\}$, and $c/f = \{2, 3\}$. This table reports the solution value (UB) and the lower bound (LB) found by CPLEX, as well as the objective value (Objective) generated by EDA-QALNS, which was executed only once for each test problem. Specifically, the optimal results are shown in bold, while the results obtained by EDA-QALNS that outperform CPLEX are both bolded and underlined.

As shown in Table 5, CPLEX can obtain optimal solutions of 15 test problems within 3600 s, where the solution values reach the lower bounds. Meanwhile, EDA-QALNS can provide optimal solutions to these problems within much shorter computation time. Additional

Table 6

Experiment results of EDA, ALNS, EDA-ALNS, and EDA-QALNS for large-sized DHFSSVRP-FEs.

Problem No.	Problem size $f \times n \times c$	EDA		ALNS		EDA-ALNS		EDA-QALNS	
		ARE	SD	ARE	SD	ARE	SD	ARE	SD
L01	$2 \times 24 \times 6$	1.89	0.95	0.95*	0.58	0.38*	0.23	0.28*	0.12
L02	$2 \times 24 \times 8$	4.42	1.95	4.82	1.47	2.82*	0.93	1.01*	0.33
L03	$2 \times 24 \times 10$	9.66	3.76	6.80	3.03	3.29	1.28	1.49*	0.48
L04	$2 \times 28 \times 6$	4.08	1.98	0.89*	0.43	0.54*	0.21	0.00*	0.00
L05	$2 \times 28 \times 8$	6.75	2.82	5.81	4.44	3.54	1.48	1.42*	0.58
L06	$2 \times 28 \times 10$	11.53	4.55	8.95	2.97	4.06	1.41	1.83*	0.42
L07	$2 \times 32 \times 6$	4.05	1.87	1.04*	0.42	1.84*	0.75	0.16*	0.05
L08	$2 \times 32 \times 8$	6.91	3.07	6.89	3.21	2.85	0.92	0.83*	0.22
L09	$2 \times 32 \times 10$	13.78	4.96	10.34	4.12	4.13	1.22	1.64*	0.69
L10	$3 \times 36 \times 9$	7.02	3.07	4.28	2.02	2.69	1.33	1.63*	0.64
L11	$3 \times 36 \times 12$	13.41	4.65	11.92	5.51	3.34	1.72	1.44*	0.51
L12	$3 \times 36 \times 15$	14.35	5.12	13.85	6.37	4.64	2.05	1.96*	0.73
L13	$3 \times 42 \times 9$	9.53	4.03	6.68	3.23	2.15*	1.02	0.97*	0.46
L14	$3 \times 42 \times 12$	11.42	4.06	8.51	3.97	5.05	2.03	2.78*	0.86
L15	$3 \times 42 \times 15$	15.53	5.78	13.08	5.19	4.83	1.71	2.23*	1.05
L16	$3 \times 48 \times 9$	8.48	3.59	5.92	2.52	3.05	1.41	2.02*	0.82
L17	$3 \times 48 \times 12$	12.55	5.12	9.54	4.61	3.51	1.23	1.59*	0.71
L18	$3 \times 48 \times 15$	18.40	7.46	14.52	6.38	5.12	2.03	2.68*	1.16
L19	$4 \times 48 \times 12$	11.86	4.92	7.68	3.69	4.11	1.56	2.02*	0.71
L20	$4 \times 48 \times 16$	13.58	5.51	9.54	4.25	5.22	2.09	2.65*	0.85
L21	$4 \times 48 \times 20$	15.25	6.63	9.67	4.66	4.82	1.76	3.03*	1.09
L22	$4 \times 56 \times 12$	11.71	4.92	9.41	4.02	3.61	1.43	2.09*	0.82
L23	$4 \times 56 \times 16$	14.37	5.71	12.86	6.05	4.22	1.85	2.35*	0.83
L24	$4 \times 56 \times 20$	18.14	7.75	16.86	7.03	5.31	1.75	2.54*	1.02
L25	$4 \times 64 \times 12$	16.68	6.45	11.81	4.78	4.08	1.97	1.75*	0.52
L26	$4 \times 64 \times 16$	18.39	8.46	14.40	7.05	6.07	2.85	3.42*	0.96
L27	$4 \times 64 \times 20$	22.25	9.24	18.23	10.56	7.16	3.25	3.26*	1.31
Average		11.70	4.75	9.08	4.17	3.79	1.54	1.82	0.66

**Fig. 5.** Factor level trend of EDA-QALNS.

preliminary experiments further indicate that removing any single destroy or repair operator, as detailed in Section 4.3, prevents the EDA-QALNS from finding these optimal solutions shown in Table 5. This highlights that the integration of these operators is crucial for the algorithm's effectiveness. For the rest of the test problems, namely S12,

S16, and S18, the solutions of EDA-QALNS are considerably better than the upper bounds obtained by CPLEX. Therefore, the proposed EDA-QALNS performs better than CPLEX for small-sized DHFSSVRP-FEs.

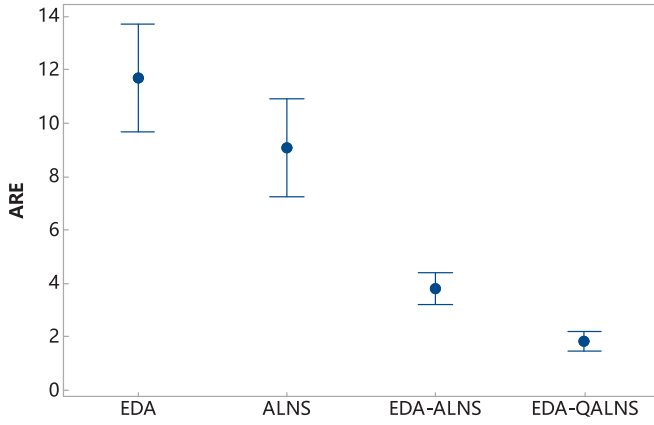


Fig. 6. The means plot with LSD intervals at a 95% confidence level for EDA, ALNS, EDA-ALNS, and EDA-QALNS.

5.4. Experiment results for large-sized DHFSSVRP-FEs

The performance of EDA-QALNS is further evaluated based on a set of large-sized DHFSSVRP-FEs, where $f = \{2, 3, 4\}$, $n/f = \{12, 14, 16\}$, and $c/f = \{3, 4, 5\}$. Because EDA-QALNS is a hybridization of EDA and ALNS, we compared it with the following potentially competitive *meta*-heuristics:

- EDA: As a standard EDA, this algorithm applies the probabilistic model to generate solutions, which is detailed in Algorithm I.
- ALNS: This algorithm is a standard ALNS. It selects the best solution from the initial population of EDA-QALNS as the initial solution, which is iteratively improved using the same destroy and repair operators of EDA-QALNS.
- EDA-ALNS: This algorithm is the same as the proposed EDA-QALNS, except for selecting the pair of destroy-and-repair operators using the well-known roulette wheel selection method of the standard ALNS.

For each of the large-sized DHFSSVRP-FEs, all the compared *meta*-heuristics are executed 20 times and terminated when the CPU time reaches $f \times n \times c \times 100ms$. Since the relative error (RE) is widely used for the performance measure of *meta*-heuristics in the literature, it is applied

to compare EDA-QALNS with the above three *meta*-heuristics as follows:

$$RE_{ir} = \frac{Z_{ir} - Z_{best}}{Z_{best}} \times 100 \quad (37)$$

where Z_{ir} denotes the objective value obtained in the r th replication by *meta*-heuristic i , and Z_{best} indicates the best objective value of all the compared *meta*-heuristics. The performance of a *meta*-heuristic is evaluated by the average of RE (ARE) and the standard deviation of RE (SD). In particular, the lower the ARE and SD, the better result the *meta*-heuristic produces.

Table 6 shows the experiment results of four *meta*-heuristics, in which ARE and SD over 20 replications are provided, respectively. The symbol ‘*’ indicates that the corresponding algorithm achieved the best solution among all compared algorithms. From this table, we can draw the following conclusions: (1) Both EDA-QALNS and EDA-ALNS perform better than EDA and ALNS in terms of ARE and SD, indicating the effectiveness of the integration of EDA and ALNS for production scheduling and vehicle routing in a multi-factory environment. (2) Compared with EDA-ALNS, EDA-QALNS provides less ARE and SD. Such good performance implies that the Q-learning algorithm is capable of selecting more efficient pairs of destroy-and-repair operators for local search than the traditional roulette wheel selection method. (3) Among the four metaheuristics, EDA-QALNS consistently generates the best solution for each test instance, whereas EDA fails to obtain the best solution in any instance, and ALNS and EDA-ALNS achieve the best solutions in only a few instances. This clearly demonstrates the superior performance and robustness of EDA-QALNS.

To statistically compare the performance of the four *meta*-heuristics, Fig. 6 shows the means plot with LSD intervals ($\alpha = 0.05$). Because the interval of EDA-QALNS is closer to zero and has no overlap with the intervals of the other three *meta*-heuristics, EDA-QALNS statistically performs the best for large-sized DHFSSVRP-FEs. Fig. 7 further illustrates the convergence of EDA, ALNS, EDA-ALNS, and EDA-QALNS for the large-sized instance L23. From this figure, we can observe that the curve of EDA-QALNS consistently lies below those of the other three *meta*-heuristics. It shows a rapid decline in the objective value at the early stage and gradually converges to the best-found solution in the later stage. This superior convergence of EDA-QALNS can be attributed to the integration of EDA with Q-learning-based ALNS.

To investigate the effect of the problem size on the performance of these *meta*-heuristics, Fig. 8 depicts the changing trends of ARE for

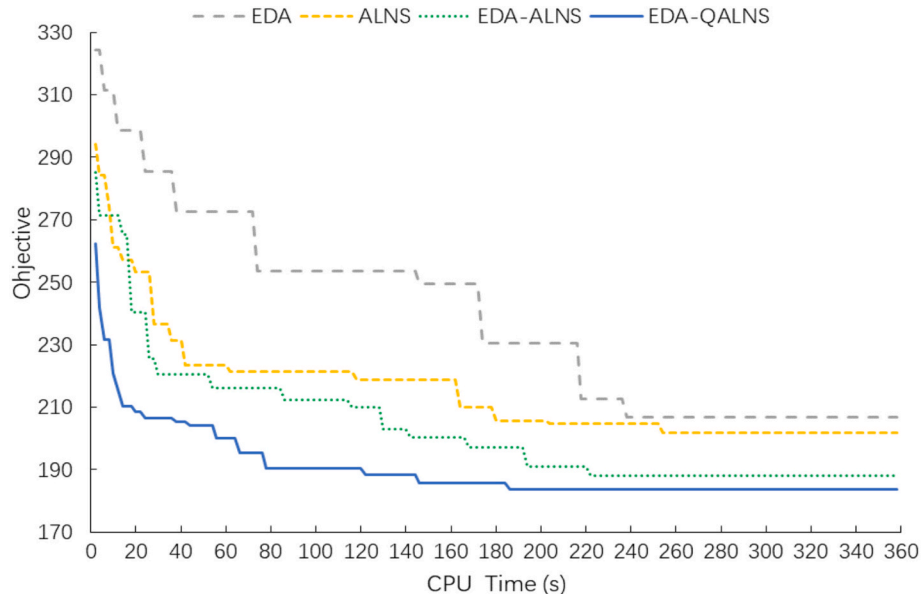


Fig. 7. The convergence plot for the large-sized instance L23.

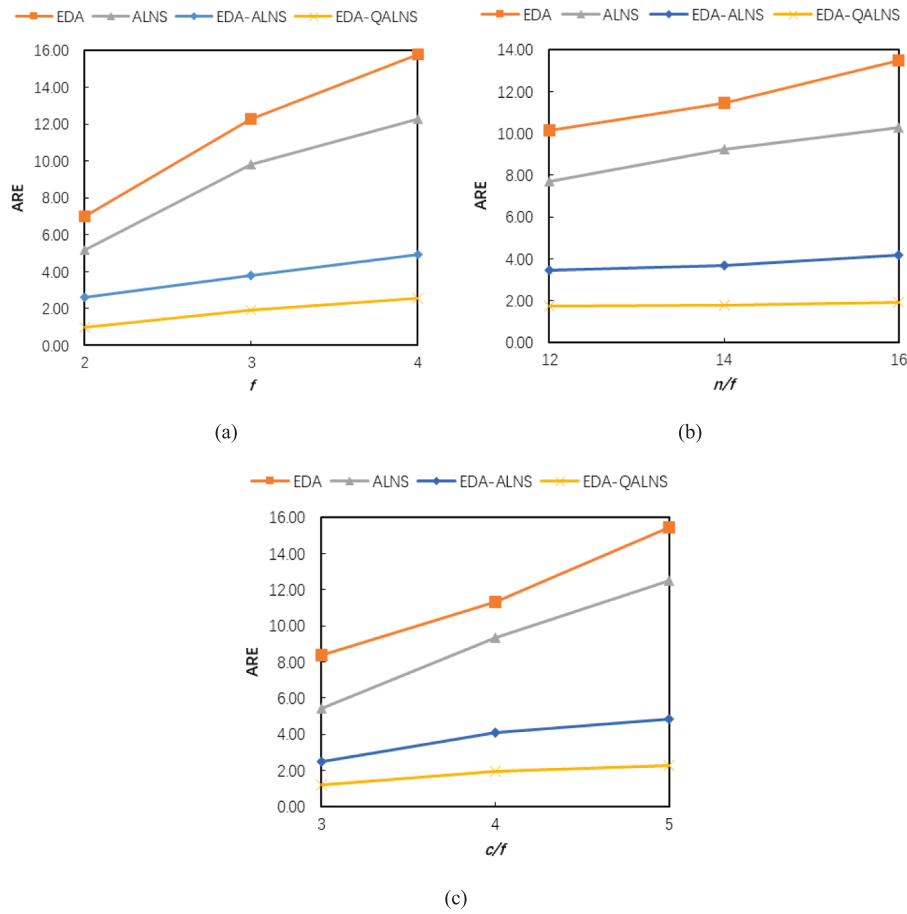


Fig. 8. Changing trends of ARE for EDA, ALNS, EDA-ALNS, and EDA-QALNS: (a) ARE against f , (b) ARE against n/f , and (b) ARE against c/f .

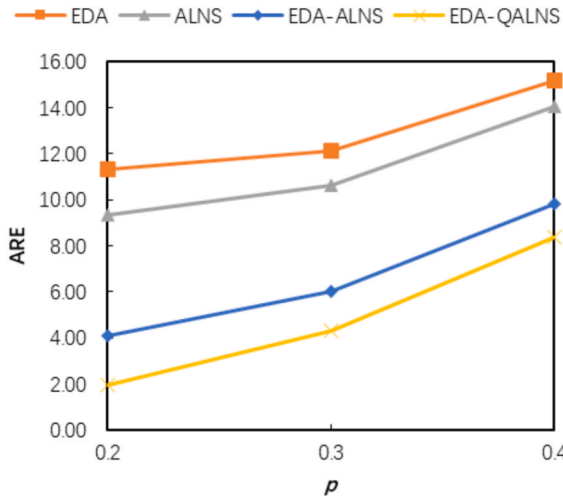


Fig. 9. Changing trends of ARE for EDA, ALNS, EDA-ALNS, and EDA-QALNS under different proportions of restricted jobs.

different sizes of f , n/f , and c/f . As shown in this figure, EDA-QALNS provides the least AREs with a slightly increasing trend. In addition, the ARE difference between EDA-QALNS and the other three meta-heuristics significantly increases when f , n/f , or c/f becomes greater. Therefore, EDA-QALNS is capable of generating the best solutions with good robustness among the four meta-heuristics, especially for large-sized DHFSSVRP-FEs.

To further examine the impact of the proportion of restricted jobs on

the performance of the compared meta-heuristics, Fig. 9 presents the changing trends of ARE with different values of p . The ARE in this figure denotes the average RE with respect to the minimum objective value obtained among all algorithms when $p = 0.1, 0.2, 0.3$. Since a larger proportion of restricted jobs inevitably reduces the search space of solutions, the performance of all compared meta-heuristics deteriorates as p increases, as indicated in Fig. 9. Furthermore, EDA-QALNS consistently achieves the best performance under different proportions of restricted jobs.

6. Conclusion

This paper focuses on a new integrated multi-factory production scheduling and vehicle routing problem (DHFSSVRP-FE), in which customer orders are firstly assigned and manufactured in several distributed HFSs with consideration of factory eligibility, and then transported to customers by capacitated vehicles within customer time windows. Compared with the traditional integrated production and distribution problem (IPDP), simultaneous consideration of factory allocation, distributed HFS scheduling, and vehicle assignment and routing in the studied DHFSSVRP-FE makes it considerably difficult to solve. Therefore, by taking advantage of EDA and ALNS, we integrate these two algorithms to develop a hybrid meta-heuristic (EDA-QALNS) for a better trade-off between search exploration and exploitation. To further improve the local search ability of ALNS, EDA-QALNS applies a novel Q-learning-based operator selection mechanism (QOSM) rather than the well-known roulette wheel selection method to determine the destroy-and-repair operators of ALNS.

To evaluate the performance of the EDA-QALNS, both small-sized and large-sized DHFSSVRP-FEs are considered. Compared with CPLEX,

EDA-QALNS generates the optimal solutions with less computation time or obtains better results than the upper bounds of CPLEX for small-sized DHFSSVRP-FEs. Furthermore, EDA-QALNS provides better solutions than three competitive *meta*-heuristics in terms of both solution quality and robustness for large-sized DHFSSVRP-FEs. The superiority of EDA-QALNS results from the integration of EDA, ALNS, and Q-learning for problem-solving. Therefore, the proposed EDA-QALNS is of great academic and practical potential to efficiently coordinate the production and distribution in the make-to-order (MTO) supply chain.

Future research may focus on developing more effective and efficient algorithms for solving the DHFSSVRP-FEs. Although three complicated and interrelated decisions have to be dealt with in the DHFSSVRP-FEs, exact algorithms, such as the branch-and-cut algorithm and the branch-and-price algorithm, can be considered to optimally solve small-sized problems. Other *meta*-heuristics can also be designed for even larger problems. In addition, the proposed model and algorithm can be further evaluated using real-world production and transportation data under more realistic customer distributions, which would enable a more comprehensive assessment of their robustness and help narrow the gap between theoretical research and practical applications in MTO industries. Furthermore, additional real-world conditions, such as machine eligibility, heterogeneous vehicles, and unexpected events in the production and distribution stages, can be incorporated to further enhance practical relevance.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We would like to thank the anonymous referees for their constructive comments. This research is supported by National Natural Science Foundation of China (No. 72171179; No. 72571201) and the Major Program of National Social Science Foundation of China (No. 22&ZD142; No. 23&ZD051).

Data availability

The data that has been used is confidential.

References

- Armstrong, R., Gao, S., & Lei, L. (2008). A zero-inventory production and distribution problem with a fixed customer sequence. *Annals of Operations Research*, 159(1), 395–414.
- Behnamian, J., & Fatemi Ghomi, S. M. T. (2016). A survey of multi-factory scheduling. *Journal of Intelligent Manufacturing*, 27(1), 231–249.
- Berghman, L., Kergosien, Y., & Billaut, J. C. (2023). A review on integrated scheduling and outbound vehicle routing problems. *European Journal of Operational Research*, 311(1), 1–23.
- Cai, J., Lei, D., Wang, J., & Wang, L. (2022). A novel shuffled frog-leaping algorithm with reinforcement learning for distributed assembly hybrid flow shop scheduling. *International Journal of Production Research*, 1–19.
- Ceberio, J., Mendiburu, A., & Lozano, J. A. (2022). A roadmap for solving optimization problems with estimation of distribution algorithms. *Natural Computing*, 1–15.
- Cheng, B. Y., Leung, J. Y. T., & Li, K. (2015). Integrated scheduling of production and distribution to minimize total cost using an improved ant colony optimization method. *Computers & Industrial Engineering*, 83, 217–225.
- Dai, M., Zhang, Z., Giret, A., & Salido, M. A. (2019). An enhanced estimation of distribution algorithm for energy-efficient job-shop scheduling problems with transportation constraints. *Sustainability*, 11(11), 3085.
- Deng, L., Di, Y., & Wang, L. (2023). A reinforcement-learning-based 3-D estimation of distribution algorithm for fuzzy distributed hybrid flow-shop scheduling considering on-time-delivery. *IEEE transactions on cybernetics*, 54(2), 1024–1036.
- Du, Y., Li, J. Q., Luo, C., & Meng, L. L. (2021). A hybrid estimation of distribution algorithm for distributed flexible job shop scheduling with crane transportations. *Swarm and Evolutionary Computation*, 62, Article 100861.
- Du, Y., Li, J. Q., Chen, X. L., Duan, P. Y., & Pan, Q. K. (2022). Knowledge-based reinforcement learning and estimation of distribution algorithm for flexible job shop scheduling problem. *IEEE Transactions on Emerging Topics in Computational Intelligence*.
- Elshaer, R., & Awad, H. (2020). A taxonomic review of metaheuristic algorithms for solving the vehicle routing problem and its variants. *Computers & Industrial Engineering*, 140, Article 106242.
- Fortes, A., Camargo, R., Muniz, L. R., de Souza Lima, F. M., & dos Reis Cota, F. (2022). Efficient matheuristics to solve a rich production-routing problem. *Computers & Industrial Engineering*, 171, Article 108369.
- Fu, Y., Hou, Y., Chen, Z., Pu, X., Gao, K., & Sadollah, A. (2022). Modelling and scheduling integration of distributed production and distribution problems via black widow optimization. *Swarm and Evolutionary Computation*, 68, Article 101015.
- Gharai, A., & Jolai, F. (2019). A branch and price approach to the two-agent integrated production and distribution scheduling. *Computers & Industrial Engineering*, 136, 504–515.
- Hauschild, M., & Pelikan, M. (2011). An introduction and survey of estimation of distribution algorithms. *Swarm and evolutionary computation*, 1(3), 111–128.
- Hou, Y., Fu, Y., Gao, K., Zhang, H., & Sadollah, A. (2022). Modelling and optimization of integrated distributed flow shop scheduling and distribution problems with time windows. *Expert Systems with Applications*, 187, Article 115827.
- Jamili, N., Ranjbar, M., & Salari, M. (2016). A bi-objective model for integrated scheduling of production and distribution in a supply chain with order release date restrictions. *Journal of Manufacturing Systems*, 40, 105–118.
- Karaoglan, I., & Kesen, S. E. (2017). The coordinated production and transportation scheduling problem with a time-sensitive product: A branch-and-cut algorithm. *International Journal of Production Research*, 55(2), 536–557.
- Kumar, R., Ganapathy, L., Gokhale, R., & Tiwari, M. K. (2020). Quantitative approaches for the integration of production and distribution planning in the supply chain: A systematic literature review. *International Journal of Production Research*, 58(11), 3527–3553.
- Li, M., Lei, D., & Cai, J. (2019). Two-level imperialist competitive algorithm for energy-efficient hybrid flow shop scheduling problem with relative importance of objectives. *Swarm and Evolutionary Computation*, 49, 34–43.
- Li, Y., Li, X., Gao, L., & Meng, L. (2020). An improved artificial bee colony algorithm for distributed heterogeneous hybrid flowshop scheduling problem with sequence-dependent setup times. *Computers & Industrial Engineering*, 147, Article 106638.
- Li, Z., Yu, H., & Zhou, S. (2023). Exact and heuristic solution approaches for integrated production and distribution problem of perishable products with customization. *Journal of the Operational Research Society*, 74(7), 1654–1669.
- Liu, H., Guo, Z., & Zhang, Z. (2021). A hybrid multi-level optimisation framework for integrated production scheduling and vehicle routing with flexible departure time. *International Journal of Production Research*, 59(21), 6615–6632.
- Mara, S. T. W., Norcahyo, R., Jodiawan, P., Lusiantoro, L., & Rifai, A. P. (2022). A survey of adaptive large neighborhood search algorithms and applications. *Computers & Operations Research*, 105903.
- Martins, L. D. C., Gonzalez-Neira, E. M., Hatami, S., Juan, A. A., & Montoya-Torres, J. R. (2021). Combining production and distribution in supply chains: The hybrid flow-shop vehicle routing problem. *Computers & Industrial Engineering*, 159, Article 107486.
- Mazyavkina, N., Sviridov, S., Ivanov, S., & Burnaev, E. (2021). Reinforcement learning for combinatorial optimization: A survey. *Computers & Operations Research*, 134, Article 105400.
- Mohammadi, S., Al-e-Hashem, S. M., & Reikik, Y. (2020). An integrated production scheduling and delivery route planning with multi-purpose machines: A case study from a furniture manufacturing company. *International Journal of Production Economics*, 219, 347–359.
- Moons, S., Ramaekers, K., Caris, A., & Arda, Y. (2017). Integrating production scheduling and vehicle routing decisions at the operational decision level: A review and discussion. *Computers & Industrial Engineering*, 104, 224–245.
- Moon, I., Lee, J. H., & Seong, J. (2012). Vehicle routing problem with time windows considering overtime and outsourcing vehicles. *Expert Systems with Applications*, 39(18), 13202–13213.
- Mühlenbein, H., & Paass, G. (1996). From recombination of genes to the estimation of distributions I. Binary parameters. In *In International conference on parallel problem solving from nature* (pp. 178–187).
- Qin, H., Li, T., Teng, Y., & Wang, K. (2021). Integrated production and distribution scheduling in distributed hybrid flow shops. *Memetic Computing*, 13(2), 185–202.
- Qiu, F., Geng, N., & Wang, H. (2022). An improved memetic algorithm for integrated production scheduling and vehicle routing decisions. *Computers & Operations Research*, 106127.
- Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, 40(4), 455–472.
- Shaw, P. (1998). Using constraint programming and local search methods to solve vehicle routing problems. In *International conference on principles and practice of constraint programming* (pp. 417–431). Berlin, Heidelberg: Springer.
- Shakya, A. K., Pillai, G., & Chakrabarty, S. (2023). Reinforcement learning algorithms: A brief survey. *Expert Systems with Applications*, 231, Article 120495.
- Sun, X. T., Chung, S. H., & Chan, F. T. (2015). Integrated scheduling of a multi-product multi-factory manufacturing system with maritime transport limits. *Transportation Research Part E: Logistics and Transportation Review*, 79, 110–127.
- Toptal, A., & Sabuncuoglu, I. (2010). Distributed scheduling: A review of concepts and applications. *International Journal of Production Research*, 48(18), 5235–5262.
- Ullrich, C. A. (2013). Integrated machine scheduling and vehicle routing with time windows. *European Journal of Operational Research*, 227(1), 152–165.
- Viergutz, C., & Knust, S. (2014). Integrated production and distribution scheduling with lifespan constraints. *Annals of Operations Research*, 213(1), 293–318.

- Wang, D., Zhu, J., Wei, X., Cheng, T. C. E., Yin, Y., & Wang, Y. (2019). Integrated production and multiple trips vehicle routing with time windows and uncertain travel times. *Computers & Operations Research*, 103, 1–12.
- Wang, J., Wang, L., & Han, H. (2025). A knowledge-driven cooperative coevolutionary algorithm for integrated distributed production and transportation scheduling problem. *IEEE Transactions on Automation Science and Engineering*, 22, 7435–7448.
- Wang, J., Lei, D., & Li, M. (2022). A Q-learning-based artificial bee colony algorithm for distributed three-stage assembly scheduling with factory eligibility and setup times. *Machines*, 10(8), 661.
- Wang, K., Choi, S. H., & Lu, H. (2015). A hybrid estimation of distribution algorithm for simulation-based scheduling in a stochastic permutation flowshop. *Computers & Industrial Engineering*, 90, 186–196.
- Wang, K., Huang, Y., & Qin, H. (2016). A fuzzy logic-based hybrid estimation of distribution algorithm for distributed permutation flowshop scheduling problems under machine breakdown. *Journal of the Operational Research Society*, 67(1), 68–82.
- Watkins, C. J., & Dayan, P. (1992). Q-learning. *Machine learning*, 8(3), 279–292.
- Xiong, F., Jing, L., Xiang, C., & Han, Y. (2025). Order acceptance and scheduling under eligibility, availability, and budget constraints in distributed heterogeneous flow shop production. *Expert Systems with Applications*, 280, Article 127586.
- Zhang, Z., Luo, Z., Baldacci, R., & Lim, A. (2021). A Benders decomposition approach for the multivehicle production routing problem with order-up-to-level policy. *Transportation Science*, 55(1), 160–178.
- Zhang, K., & Zhou, B. (2024). A self-adaptive Gaussian mutation-based arithmetic optimiser algorithm for integrated production scheduling and vehicle routing problem in the distributed manufacturing environment. *International Journal of Production Research*, 62(21), 7952–7980.
- Zhao, F., Shao, Z., Wang, J., & Zhang, C. (2016). A hybrid differential evolution and estimation of distribution algorithm based on neighbourhood search for job shop scheduling problems. *International Journal of Production Research*, 54(4), 1039–1060.
- Zhao, S., & Zhou, H. (2025). Learning-driven memetic algorithm for solving integrated distributed production and transportation scheduling problem. *Swarm and Evolutionary Computation*, 96, Article 101945.